Individual Contribution Reflection

Allen Ji (560332702)

COMP4347 / COMP5347 Assignment 2 - MERN Quiz Game

Subsystem: Admin Question Management

---

1. Subsystem ownership. I owned the admin question-management subsystem: protected admin APIs, question create/edit/delete, active/inactive toggle, bulk import, admin dashboard UI, modal question form, and admin validation. This subsystem lets admins maintain the question bank used by the player quiz and Review Mode explanations.

2. Major technical challenge. The main technical challenge was validating flexible admin input without weakening the fixed quiz mechanics. Bulk import accepts a JSON textarea, but every question must still have non-empty text, exactly four options, a correctAnswer index from 0 to 3, active state, and optional explanation/topic fields. I handled this with frontend Zod parsing, backend validation, clear per-item error messages, and protected routes that reject non-admin users.

3. Mermaid sequence diagram.

```
sequenceDiagram
  participant Admin as Admin UI
  participant Form as Question Form/Bulk Import
  participant API as Admin Routes
  participant Guard as JWT + Admin Middleware
  participant Q as Question Model
  Admin->>Form: Create, edit, toggle, import
  Form->>API: POST/PUT/PATCH/DELETE /api/admin/questions
  API->>Guard: verify JWT and role=admin
  Guard->>Q: validate and mutate question bank
  Q-->>API: saved question data
  API-->>Form: success/error envelope
  Form-->>Admin: refreshed list and feedback
```

4. Review Mode design reflection. The approved variation requires explanations in Review Mode, so the admin subsystem supports optional explanation and topic fields while preserving the base rule of exactly four options and one correct answer. Active/inactive status also lets admins manage question availability without deleting historical attempts.

5. Cross-system understanding. This subsystem depends on the full MERN stack: React collects input and displays feedback, Express exposes protected REST routes, middleware enforces role and validation rules, Mongoose persists the relevant state, and the shared response envelope keeps frontend error handling predictable.

6. Git commit history analysis. The table below cites meaningful commits from the GitHub Enterprise history and explains what each contributed technically.

Commit hashes: `2479978 df3c0cc fc0f700 284fd25 63e5093 1768b9e ee4e681 01a0100 a480298 c89b303 3a77060 c8b48f6 1c2d468 5f88633`

Hash	Focus	Evidence
`2479978`	Admin API	Added protected question controller and admin routes.
`df3c0cc`	Admin UI	Added dashboard, question form, and bulk import.
`fc0f700`	Admin styling	Aligned admin page styles with existing CSS.
`284fd25`	Admin tests	Added admin controller tests for create/import/toggle/access.
`63e5093`	Frontend alignment	Aligned JSX context and placeholder file extensions.
`1768b9e`	Page imports	Fixed frontend page imports and JSX names.
`ee4e681`	Seed alignment	Aligned seed data with Question schema.
`01a0100`	Routing	Resolved route imports and quiz start navigation.
`a480298`	Modal UX	Displayed question form in an admin modal.
`c89b303`	Route security	Secured quiz submission and leaderboard routes.
`3a77060`	Admin login	Separated admin login from public quiz entry.
`c8b48f6`	Consistency	Improved admin route consistency and quiz validation.
`1c2d468`	Status codes	Returned correct API status codes.
`5f88633`	Submission cleanup	Finalized submission documents and removed stale files.

7. Final reflection. My contribution was not only a list of commits; it was a bounded subsystem responsibility with evidence. The commits above show implementation, validation, integration, and final submission hardening. In the demo I can explain both my own files and how my subsystem interacts with authentication, quiz attempts, admin data, Review Mode, and MongoDB persistence.